



Memória + *higiene de contexto*.

Em *15min* você entende como seu agente lembra de tudo — daily notes automáticas, memória semântica que busca por *significado*, e como você ajuda ele a manter organizado pra escalar com você.

Memória sem higiene = agente lerdo. Higiene sem memória = começar do zero todo dia.

A-HA DESTA AULA

A-HA #5 ENTREGUE

"Ele lembra de mim e das decisões."

Persistência real. Aluno PME para de pensar "*começar do zero todo dia*" e entende que workspace + 3 tipos de memória = agente com história contínua. Não é mágica · é **arquitetura**.

PRE-FLIGHT CHECK

Antes de *apertar play*.

A7 é modular, mas tem dependência crítica: memória semântica só roda se você configurou a OpenAI API key na A1. Sem isso, metade da aula vira teoria sem prática.

- ✓ **OpenAI API key configurada** (A1 Fase A) — embeddings rodam ali · sem ela, memória semântica não funciona
- ✓ **Workspace + MAPAs** (A6) — daily notes em `memory/YYYY-MM-DD.md`, MAPAs orientam organização
- ✓ **Identidade aprofundada** (A5) — agente vai consultar SOUL+IDENTITY+USER no boot
- ✓ **15min sem interrupção** — 3 conceitos técnicos (boot · semântica · higiene) merecem atenção
- ✓ **Telegram aberto** — Bruno demonstra "esquecer e lembrar" ao vivo no Bloco 6

⚠ **Não configurou API key na A1?** Volta lá. Memória semântica é o que faz o agente buscar por *significado* em vez de palavra exata — sem ela, ele responde só com daily notes recentes (48h) e MEMORY.md, perde busca histórica.

ESTRUTURA DA AULA EM VÍDEO

O que você vai aprender em *7 blocos*.

Princípio transversal (continuado de A5+A6): **Telegram aberto em paralelo**. Bruno demonstra "esquecer e lembrar" ao vivo — sessão zerada, agente respondendo com contexto preciso.

🕒 3 MIN

Bloco 1 — Mecânica concreta primeiro

Sem teoria de saída. Daily notes em `memory/YYYY-MM-DD.md` · salvamento direcionado "*salva isso importante*" · **Memory Flush** antes da compactação · confiabilidade real **~70%**. Por isso você ajuda salvando manual quando importa.

🕒 2.5 MIN

Bloco 2 — 3 tipos de memória

Padrão (boot) — agente acorda lendo MEMORY.md + dailies 48h + MAPAs.

Sessão — chat atual. **Semântica** — busca por significado (auto-ativa com API key da A1).

🕒 2 MIN

Bloco 3 — Sessão + sintomas + sessions.json

Janela 200k tokens. Sintomas de saturação: lento · alucinação · perde contexto. Causa comum: muitos tópicos no Telegram. Tip: `/status` ao vivo. Path real: `~/.openclaw/agents/<id>/sessions/sessions.json`.

🕒 2.5 MIN

Bloco 4 — Memória semântica

Busca por **significado**, não palavra. Auto-ativada com API key da A1. Comandos manuais Tier 2: `openclaw memory status` · `search` · `index --force`. Custo: centavos por indexação completa.

🕒 2.5 MIN

Bloco 5 — Higiene + comparação Claude Code

Coração da aula. Aluno vindo do Claude reconhece tudo. Comandos nativos: `/compact` · `/new` · `/reset` · `/status`. Princípio absoluto: **"salve antes de limpar"**.

🕒 1.5 MIN

Bloco 6 — Demo: ele lembra (a-ha)

Sessão zerada (chat novo). Aluno pergunta sobre decisão antiga · Aurora responde com contexto preciso citando data + raciocínio. *"Você fala uma vez, ele lembra pra sempre."*

🕒 1 MIN

Bloco 7 — Recap + amarração A8

Recap dos 3 tipos · Memory Flush 70% · comandos nativos · "salve antes de limpar". Amarração: *"Memória ele tem. Próxima aula é Skills (A8) — capacidades modulares."*

APROFUNDAMENTO DO BLOCO 1

Mecânica primeiro — *como ele escreve.*

Antes de teoria. Como agente OpenClaw **escreve memória** no day a day — e onde você participa.

Daily notes — fluxo automático

Toda atividade significativa do dia é capturada em `memory/YYYY-MM-DD.md`.
Markdown puro, padrão OpenClaw. Solta na raiz de `memory/` (recap A6, não em `sessions/`).

- ✓ **Quando agente escreve:** ao longo da conversa, no final do dia, ou quando você pede explicitamente
- ✓ **O que captura:** decisões, calls, pendências, contexto novo, resultados de tarefas executadas
- ✓ **Formato:** bullet points + timestamp · não enche linguiça · captura essência
- ✓ **Quem lê depois:** próprio agente no boot do dia seguinte · você quando consulta

Salvamento direcionado – quando você ajuda

Você pode pedir explicitamente que ele salve algo importante. E pode **direcionar onde**:



PROMPT · SALVAMENTO DIRECIONADO

COLA NO TELEGRAM

salva isso importante: [DECISÃO / CONTEXT0]

Salva tanto na daily note de hoje quanto no lugar estruturado que faz sentido (ex: pasta do projeto, context/decisoões/, ou context/people/{nome}.md se for sobre alguém específico).

Me mostra onde você salvou.



Salve o contexto atualizado de todos os projetos e decisões importantes em uma pasta chamada context/decisoões/ ou context/people/{nome}.md

Memory Flush – feature nativa OpenClaw

Conceito técnico importante: antes da compactação automática disparar, agente faz um **"silent memory flush"** – tenta salvar nas pastas certas antes de comprimir. Documentado em docs.openclaw.ai/concepts/compaction.



Confiabilidade real (estimativa Bruno): ~70% das vezes funciona limpo. Compactação às vezes mata o salvamento – token budget acaba antes do flush completar. **Por isso você ajuda salvando manual quando o assunto importa. Não confia 100% no flush automático.**

FRASE DO BLOCO 1

"Memória do agente é simples: ele escreve toda hora. Você ajuda quando importa."

3 tipos de memória *trabalhando juntos*.

Agente OpenClaw tem 3 sistemas de memória que conversam entre si. Cada um cobre coisa diferente. Não substituem — complementam.

Tabela comparativa

TIPO	O QUE COBRE	QUANDO ATIVA
1. Padrão (boot)		
2. Sessão		
3. Semântica		

1. Memória padrão · *como ele acorda*

Quando agente "acorda" numa nova sessão (você abre o Telegram, manda mensagem nova depois de horas/dias), ele **automaticamente**:

- ✓ Lê `MEMORY.md` — long-term self-knowledge (recap A6)
- ✓ Lê **daily notes de hoje + ontem** — 48h de contexto (padrão OpenClaw)
- ✓ **Carrega MAPAs do workspace** — sabe onde tá cada tipo de coisa (Princípio 12, recap A6)
- ✓ **Resultado:** ele acorda já sabendo de tudo que tá rolando — não começa do zero

2. Memória de sessão · *chat atual*

Janela do contexto da conversa em curso. Tem limite — 200k tokens com GPT-5.5 (~150 páginas de texto). Cobre Bloco 3 abaixo.

3. Memória semântica · *busca por significado*

O sistema mais poderoso. Cobre Bloco 4 abaixo. **Já vem ativada quando você configurou OpenAI API key em A1** — não precisa skill nova nem setup adicional.

i **Por que precisava da API key na A1:** embeddings (vetores numéricos que representam significado) rodam via OpenAI direto. OAuth da Fase B cobre raciocínio · API key da Fase A cobre Whisper + DALL-E + **embeddings da memória semântica**. Cada credencial tem papel específico.

APROFUNDAMENTO EXTRA · CONTEXTO QUENTE

`memory/hot.md` — o *resumo do agora*.

Os 3 tipos cobrem boot, sessão e busca histórica. Mas tem uma quarta peça que vive na raiz de `memory/` e merece nome próprio: `hot.md`. Pensa nele como a **tela inicial do agente** — snapshot do que importa *agora*, atualizado o tempo todo, lido em toda sessão.

O que tem dentro

- ✓ **Top prioridades da semana** — 3-5 itens que precisam de atenção real
- ✓ **Negociações ativas** — quem tá pendurado, último toque, próximo passo
- ✓ **Decisões recentes** — o que foi fechado nos últimos 7-14 dias
- ✓ **Métricas-chave** — números do mês que importam (receita, alunos, eventos)
- ✓ **Prazos críticos** — eventos com data próxima que não podem passar

Quem mantém

`hot.md` é **auto-gerenciado**. O agente atualiza no fluxo normal: quando você decide algo importante, surgem prioridades novas, ou um prazo se aproxima — ele edita. Você empurra atualização explícita com a skill `/salve` (cola final de sessão · agente lê o que aconteceu e atualiza pendências, decisões, prazos, hot). Você não escreve `hot.md` à mão.

Função 1 · *boot context*

Toda nova sessão, agente lê `hot.md` junto com MEMORY.md e dailies. Diferença: MEMORY.md é durável (auto-conhecimento que muda devagar), dailies são cronologia (o que aconteceu), `hot.md` é **estado atual** (o que importa agora). Ele

entra no flow já sabendo o que tá quente — não precisa perguntar "no que a gente tava?".

Função 2 · *combustível do heartbeat*

Aqui é onde `hot.md` conecta com A9. Heartbeat (mecanismo de proatividade · poll periódico) consulta `hot.md` a cada batida pra decidir **o que vigiar**. Sem `hot.md`, heartbeat fica perdido — checa coisa genérica que não importa pra você nessa semana.

Exemplo concreto: você fechou contrato com a Furia, prazo do primeiro entregável é *22/04*. Skill `/salve` roda · agente atualiza `hot.md` com *"prazo crítico Furia 22/04"*. A cada heartbeat (a cada 30min), agente lê `hot.md` · vê o prazo · checa se passou ou tá próximo · te avisa só se valer interromper. Sem `hot.md`, heartbeat não saberia que esse prazo existia.

Distinção importante (recap A3): `HEARTBEAT.md` (raiz do workspace) é a **configuração** do mecanismo — poll period, checks rotacionáveis, quiet hours. `memory/hot.md` é o **contexto** que esse mecanismo consulta. HEARTBEAT.md = COMO ele vigia. hot.md = O QUE ele vigia. **A9 cobre o mecanismo · essa seção cobre o contexto.**

APROFUNDAMENTO DO BLOCO 3

Sessão · sintomas + *sessions.json*.

Memória de sessão é o chat atual. Janela enorme (200k tokens GPT-5.5 ≈ 150 páginas), mas finita. Quando enche, surgem **sintomas de saturação**:

Sintomas de contexto saturado

- ✓ **Agente lento** — demora pra responder, parece "pensando demais"
- ✓ **Não responde** — trava no meio, pula, ou retorna erro
- ✓ **Alucina** — inventa contexto que não existiu, mistura conversas
- ✓ **Perde contexto antigo** — esquece o que foi falado há 30min

Causa comum (amarração A4)

Em 90% dos casos: **muitos tópicos no Telegram com o mesmo agente**. Cada tópico cria um arquivo de transcript separado. Soma fica grande, contexto satura.

```
~/openclaw/agents/<agentId>/sessions/  
├── sessions.json                ← mapa de todas as sessões  
├── <sessionId>.jsonl           ← transcript da sessão  
└── <sessionId>-topic-<threadId>.jsonl ← 1 arquivo por tópico
```



Reforça regra A4: máximo **3-4 tópicos** por agente no Telegram. Mais que isso, sessão satura rápido. Tópicos por tema (não por dia, não por humor) são a divisão saudável.

Tip prático ao vivo – diagnóstico

Se agente fica lento, pede pra ele rodar diagnóstico:



PROMPT · DIAGNÓSTICO DE CONTEXTO

COLA NO TELEGRAM

roda /status e me diz como tá o uso de contexto.

Se tá perto do limite (>70%), me sugere:

1. O que vale salvar antes de compactar
2. Se vale começar nova sessão (/new) ou compactar (/compact)
3. Se algum tópico tá inflando demais



Se salvar mais de 10 arquivos, o ML vai começar a compactar automaticamente. Se não salvar, o contexto vai saturar e o agente vai ficar lento.

Quer ver o tamanho físico? Pede:



olha o tamanho do sessions.json em
~/openclaw/agents/<id>/sessions/ e me diz se tá grande.

Lista também os arquivos -topic-*.jsonl ordenados
por tamanho – quero ver se algum tópico tá inflando.

`sessions.json``<sessionId>-topic-<threadId>.jsonl`

Higiene = não esquecer de compactar + começar nova sessão. Bloco 5
cobre os comandos exatos. Princípio: salve antes de limpar (sempre).

APROFUNDAMENTO DO BLOCO 4

Memória semântica — *busca por significado.*

Sistema que faz o agente buscar pelo **significado** do que você perguntou, não pela palavra exata. Você pergunta "lembra do que decidimos sobre WhatsApp?" e ele acha `decisoes/2026-04.md` mesmo que o arquivo não tenha a palavra "WhatsApp" no nome.

Como funciona (alto nível)

1

Indexação contínua

Agente roda embeddings de cada arquivo do workspace (memory, content, archive) e guarda os vetores. OpenAI API key da A1 cobre.

2

Você pergunta com significado

"o que decidimos sobre comunidade?" · agente vetoriza a pergunta e busca os arquivos com vetores próximos.

3

Retorna resultados ranqueados

Top arquivos por proximidade semântica. Agente lê os mais relevantes, sintetiza resposta com data + raciocínio + próximos passos.

Demo concreto

 **PROMPT · BUSCA SEMÂNTICA**

COLA NO TELEGRAM

lembra do que decidimos sobre WhatsApp?

Faz busca semântica no workspace, não busca por palavra exata. Me responde com:

1. Data da decisão
2. Raciocínio (por que decidimos isso)
3. Onde tá registrado (arquivo)
4. Se mudou desde então

 💡 Este prompt funciona melhor se você tiver o contexto de uma conversa anterior. Se não, o agente pode não entender o que você quer.

Comandos manuais (Tier 2)

Pra power users e debug. Agente faz tudo automaticamente – você roda manual só se quiser debug ou índice forçado:

 **TERMINAL MANAGED**

COMANDOS TIER 2

```
$ openclaw memory status
$ openclaw memory search "query"
$ openclaw memory index --force
```

 👉 `--force` força a reindexação de todos os arquivos, mesmo os que não foram modificados.

Custo

- ✓ **Indexação completa:** centavos pra workspace típico (poucos MB de markdown)
- ✓ **Busca individual:** fração de centavo (1 query semântica ≈ R\$ 0,01-0,02)
- ✓ **Reindexação automática:** agente faz incremental (só arquivos modificados)

💡 **Princípio:** "Sempre que você pesquisar sobre alguém, empresa, projeto — peça com detalhes. Quanto mais específico, melhor a busca semântica acha." Pergunta vaga = resposta vaga. Pergunta detalhada = resposta cirúrgica.

APROFUNDAMENTO DO BLOCO 5

Higiene de contexto — *Claude Code vs OpenClaw.*

Coração da aula. Aluno que veio do Claude Code reconhece quase tudo — OpenClaw seguiu padrão similar. Pra quem não veio, é introdução natural.

📘 **Aviso de abertura:** "Se você usa Claude Code, vai reconhecer quase tudo. OpenClaw seguiu padrão similar." Comandos com mesma cara, comportamento próximo, bonus em alguns casos.

Comparação lado a lado

MOVIMENTO	CLAUDE CODE	OPENCLAW
Compactar contexto	<code>/compact</code>	<code>/compact Focus on [tema]</code>
Nova sessão	<code>/new</code>	<code>/new</code> <code>/reset</code>
Trocar modelo na nova		<code>/new <model></code>
Ver estado	<code>/status</code>	<code>/status</code>
Salvar antes de limpar	<code>/salvar</code>	

Princípios da higiene OpenClaw

- ✓ **Princípio absoluto (recap A0):** *"Salve antes de limpar."* Memory Flush automático ajuda mas não é 100%
- ✓ **Auto-compaction ON por default** — dispara quando contexto enche ou modelo retorna context-overflow
- ✓ **Comando natural cobre 80% dos casos** — você não decora atalho · pede pro agente em português
- ✓ **Quando mudar de assunto:** dá `/new` pra começar limpo. Antes disso, salve

Comando natural — o jeito OpenClaw

Você não precisa decorar slash commands. Pede em português:



**PROMPT · SALVAR ANTES DE LIMPAR
(NATURAL)**

COLA NO TELEGRAM ANTES DE
`/NEW`

salva tudo importante dessa conversa e esquece o resto.

Em específico, quero garantir que vai pra memória persistente:

1. Decisões que tomamos (com data + raciocínio)
2. Pendências novas que surgiram
3. Contexto sobre pessoas/projetos que valha guardar
4. Próximos passos definidos

Depois eu vou abrir nova sessão pra mudar de assunto.



Salve tudo importante dessa conversa e esqueça o resto. Salve o que quiser pra memória persistente.
Depois eu vou abrir nova sessão pra mudar de assunto.

`/new`

Comandos slash · referência rápida

▶ SLASH COMMANDS NO TELEGRAM

MANDA DIRETO NA CONVERSA

```
/ status
/ compact Focus on [tema]
/ new
/ new gpt-5.5
/ reset
```

▶ Clique para ver todos os comandos de slash e como usá-los no Telegram

Manutenção Tier 2 (terminal)

Comando opcional pra power users — limpa transcripts órfãos quando você acumulou muitas sessões antigas:

▶ TERMINAL MANAGED

LIMPEZA OPCIONAL

```
$ openclaw sessions cleanup --enforce
```

▶ Clique para ver todos os comandos de terminal e como usá-los no Telegram

⚠ **Sem ChatGPT Plus / API key na A1?** Memória semântica não funciona · comandos manuais retornam erro. Volta na A1 antes de tentar A7. Tudo conecta.

BLOCO 6 · A-HA #5 AO VIVO

Demo final — *"ela lembra apesar do chat zerado"*.

Demo que entrega o a-ha. Persona **Aurora** (continuidade A5/A6, empreendedor PME). Sessão zerada visível — Bruno dá `/new` ao vivo antes da pergunta. Aluno vê o chat ficar vazio e a Aurora ainda assim responder com contexto preciso.

Cenário

- ✓ **Telegram da Aurora** — chat ativo da semana
- ✓ **/new ao vivo** — Bruno aperta · chat zera · contexto da sessão sai
- ✓ **Pergunta sobre decisão antiga** — algo que aconteceu semanas atrás, não tá no contexto da sessão
- ✓ **Aurora responde com contexto preciso** — boot lê MEMORY.md + dailies 48h · semântica busca decisão antiga · sintetiza



PROMPT · PERGUNTA APÓS /NEW TELEGRAM DA AURORA · SESSÃO ZERADA

Aurora, lembra do que a gente decidiu sobre
[DECISÃO ANTIGA — ex: contratação de Head of Marketing]?

Me responde com:

1. Data da decisão
2. Raciocínio
3. Próximos passos definidos
4. Onde tá registrado



Aluno pergunta sobre decisão antiga e pergunta se o contexto da sessão zerada não apaga a memória. Boot responde com contexto preciso e sintetiza resposta.

FRASE DO MOMENTO

"Você fala uma vez, ele lembra pra sempre. Você ajuda salvando o importante. Ele faz o resto."



A-ha emocional: aluno vê que sessão zerada não apaga memória. Boot reconstrói contexto · semântica acha histórico · agente sintetiza resposta. Persistência real, sem mágica — arquitetura.

TRAVOU? ANTES DE PEDIR AJUDA

Top *problemas comuns da A7.*

SINTOMA	CAUSA PROVÁVEL	AÇÃO
		<code>openclaw memory status</code>
		<code>openclaw memory index --force</code>
		<code>/status</code> <code>/new</code>
<code>/compact</code>		<code>/new</code>
	<code>memory/sessions/</code>	
<code>sessions.json</code>		<code>openclaw sessions cleanup --enforce</code>

SINTOMA	CAUSA PROVÁVEL	AÇÃO
/new		



Cheatsheet completo: [memoria-do-seu-agente.md](#) – 3 tipos ·

/new

 ·

/compact

 ·

/status

 · comando natural pra salvar · paths do session store · *"salve antes de limpar"*.



Não conseguiu resolver com a tabela acima? Joga screenshot/print no [grupo Telegram do curso](#). openclawzinho responde em < 30 segundos · humanos em < 5 minutos.

3

tipos de memória
(boot · sessão ·
semântica)

~70%

confiabilidade real
do
Memory Flush
automático

200k

tokens da janela
de sessão (GPT-
5.5)

48h

de daily notes
carregadas
no boot automático

FRASE-MÃE DA A7

Pra colar na *geladeira*.

"Memória sem higiene = agente lerdo. Higiene sem memória = começar do zero todo dia. Os dois juntos = ele escala com você."

3 tipos (padrão · sessão · semântica) · Memory Flush 70% (não confiar 100%) · `/compact` `/new` `/status` nativos · comando natural · *"salve antes de limpar"*.

Próximo passo: *A8 — Skills + 6 curadas + criar a sua*

14 minutos · modular · memória ele tem. Próxima aula é sobre **capacidades modulares** — skills nativas + 6 curadas pelo curso + como criar a sua. Aluno sai sabendo invocar capacidade certa pra cada cenário e ampliar o agente sem inflar o prompt principal.

→ Ir pra Aula 8

TRAVOU? TÁ NO GRUPO DO CURSO

Comunidade Telegram do curso — t.me/courseopenclaw — suporte 24/7 (openclawzinho bot + Bruno e equipe presente). Dúvida sobre essa aula, problemas de setup, ou quer compartilhar como aplicou no seu negócio: bora trocar lá.

PRÓXIMO NÍVEL: PIXEL AI HUB

Aplicar agente pra empresa toda — equipe, governance, escala? [Pixel AI Hub](#): segundo cérebro corporativo, comunidade de empreendedores, +50 skills enterprise, encontros mensais ao vivo. Vagas mensais limitadas.

